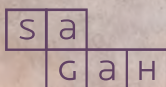


PREPARAÇÃO E ANÁLISE EXPLORATÓRIA DE DADOS

Mariana Araújo Pereira



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Normalização de dados usando Pandas, Numpy e Scikit-learn

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Identificar a necessidade de normalização de dados.
- Reconhecer o método adequado de normalização de dados, de acordo com cada variável da base de dados.
- Implementar rotinas de normalização de dados usando as ferramentas Pandas e Numpy.

Introdução

Um banco de dados pode ser constituído por variáveis de diferentes grandezas. Quando isso acontece e se deseja avaliar esses dados em conjunto, é necessário que eles sejam transformados para que tenham o mesmo intervalo de valores. Esse processo é chamado de normalização. A normalização nem sempre é necessária, mas é ideal verificar se essa necessidade existe ou não e escolher o método mais adequado de acordo com cada variável, a fim de não distorcer os dados.

Neste capítulo, você vai ver como identificar quando a normalização de dados é necessária, como escolher o método mais adequado para cada tipo de variável e como implementar esses métodos utilizando as bibliotecas Pandas e Numpy.

1 Normalização de dados

A normalização é uma etapa crucial no pré-processamento de dados. Esse processo consiste na transformação dos diferentes dados de um banco, com diferentes grandezas, a fim de torná-los comparáveis, estabelecendo uma grandeza única. Durante uma análise com dados não tratados de diferentes grandezas, pode haver uma tendência falsa, direcionada pelos valores extremos. A normalização então é utilizada para dimensionar os dados de cada variável em um intervalo pré-definido, como $-1,0$ a $1,0$ ou $0,0$ a $1,0$, de forma individualizada, garantindo que não haja distorções nas diferenças preexistentes em cada conjunto.

De acordo com Al Shalabi, Shaaban e Kasasbeh (2006), esse processo é útil principalmente para algoritmos de classificação, podendo melhorar a precisão e eficiência de algoritmos de mineração de dados, envolvendo redes neurais, vizinho mais próximo e classificadores de agrupamentos (comumente chamados de *clusters*, em inglês). Esses métodos fornecem melhores resultados se os dados a serem analisados foram normalizados, ou seja, se estiverem dimensionados em intervalos específicos.

Se pensarmos em um banco com dados de funcionários de uma empresa, que inclui a idade, os anos de experiência e o salário, podemos observar duas situações em que a normalização é necessária ou não. Quando a idade e os anos de experiência são comparados, não é necessário fazer tratamento, já que ambos têm a mesma unidade de medida (em anos), que pode variar no máximo entre 0 e 100. Já quando comparamos os anos de experiência dos funcionários e o salário de cada um deles, por exemplo, a normalização é necessária. Isso porque as grandezas entre a experiência e o salário são diferentes, já que a primeira está na ordem das dezenas enquanto a segunda está na ordem do milhar.

A escolha de normalizar ou não os dados depende, principalmente, de como esses dados serão trabalhados entre si. Se o objetivo é comparar as variáveis para fins de classificação, por exemplo, a normalização é fundamental. Isso porque nesse tipo de análise a importância de uma variável depende da sua variabilidade em relação a outras variáveis.

Se uma variável como a de salário, do exemplo anterior, for comparada à de idade, esta primeira terá uma contribuição maior no modelo. Ao normalizar ambas para um mesmo intervalo, essa relação entre valor e importância deixa de existir individualmente, diminuindo o viés da análise. Assim, o objetivo da normalização é fazer com que cada ponto de dados tenha a mesma escala para que cada recurso seja igualmente importante. No caso da importância de cada

variável ser conhecida anteriormente, é possível realizar essa normalização com intervalos distintos, mas para isso é necessário ter um conhecimento mais sólido e delinear muito bem como isso será feito.

Em um contexto distinto, no qual as variáveis são combinadas de forma linear, nem sempre é estritamente necessário normalizar as entradas. Nesse caso, modificar as escalas não alterará a correlação e desvios correspondentes, retornando o mesmo resultado com e sem normalização. Entretanto, a aplicação da normalização nesses casos pode diminuir o tempo de processamento da análise, tornando-o mais rápido.



Fique atento

Os conceitos de padronizar, redimensionar e normalizar dados, apesar de muito parecidos e algumas vezes utilizados em conjunto, são diferentes, acompanhe:

- **Padronizar** os dados de uma variável é utilizar medidas desses mesmos dados para modificá-los, criando uma distribuição normal. Pode-se, por exemplo, subtrair a média e dividir pelo desvio-padrão do conjunto de dados da variável, obtendo-se um “padrão normal”.
- **Redimensionar** uma variável significa modificar o seu valor utilizando uma ou mais constantes por meio de operações matemáticas como adição e multiplicação, a fim de alterar as unidades de medida dos dados. Um exemplo é a transformação de temperatura de graus Celsius a Fahrenheit, que pode ser obtida pela fórmula $(\text{graus } ^\circ\text{C} \times 1,8) + 32$, em que os valores 1,8 e 32 são constantes.
- **Normalizar** é modificar os dados da variável utilizando determinada norma, com o objetivo de reescalonar os dados para um intervalo pré-determinado entre 0 e 1, por exemplo, sem modificar as proporções de valores dentro daqueles dados.

Normalização no Python

Existem diferentes formas de realizar a normalização no Python. Como as fórmulas que norteiam cada técnica são conhecidas, você pode criar uma função que utilize essas fórmulas para cada variável do seu banco de dados. Além dessa forma “faça você mesmo”, existem bibliotecas que já trazem funções prontas para realizar essa normalização. Como esse é um passo importante no pré-processamento de dados, diferentes profissionais incluíram essa funcionalidade em seus pacotes, de forma que você pode encontrá-las pelo menos em uma dezena deles. Neste capítulo utilizaremos as conhecidas bibliotecas Numpy, Pandas e Scikit-learn para realizar o processo de normalização.

Numpy é uma poderosa biblioteca do Python, que suporta a manipulação de matrizes e conjuntos (ou *arrays*) multidimensionais, tendo diferentes funções matemáticas em seu pacote. **Pandas** é uma das bibliotecas mais conhecidas e utilizadas, devido a sua rapidez e capacidade de lidar com a manipulação e análise de grandes volumes de dados de forma simplificada. **Scikit-learn**, por sua vez, é uma biblioteca de aprendizado de máquina (*machine learning*) que inclui vários algoritmos de normalização, classificação, regressão e agrupamento de dados, e foi projetada para interagir com bibliotecas numéricas como Numpy. A utilização dessas três bibliotecas conjuntamente permite uma manipulação de dados de forma fácil e eficiente, utilizando das funções nelas embutidas.

2 Tipos de normalização

A normalização de dados é útil principalmente quando se utilizam algoritmos de classificação. Para que ela seja realizada, existem diferentes técnicas, que se diferenciam de acordo com o atributo que será utilizado para a modificação da escala e podem apresentar vantagens ou desvantagens, a depender das particularidades de cada banco de dados, como será exemplificado adiante. Entre as técnicas de normalização de dados, as mais comuns, de acordo com Patro e Saho (2015), são as técnicas de mínimo–máximo, *Z-score* e escala decimal.

Mínimo–máximo

A técnica de normalização considerada mais simples é a normalização mínimo–máximo. Essa técnica de normalização é mais adequada quando o intervalo (valores máximo e mínimo) dos dados de cada variável é conhecido e não há valores muito discrepantes. Além disso, pode ser utilizada quando é importante que os valores iguais a zero sejam assim preservados. Nesse caso, o valor mínimo e máximo do conjunto de dados da variável é alterado para 0 e 1, respectivamente. Os outros valores devem ser transformados em novos valores que estejam entre esses dois números, calculados com base na fórmula a seguir:

$$\frac{\text{valor} - \text{mínimo}}{\text{máximo} - \text{mínimo}}$$

O problema é que esse método é altamente sensível à presença de valores discrepantes nos dados. Se um conjunto A tem 10 valores entre 0 e 40, por exemplo, e esses valores são transformados em 0 e 1, o valor igual a 20 corresponderia a 0,5 de acordo com a fórmula, já que representa a metade dos valores entre 0 e 40. Entretanto, se um conjunto B tiver os mesmos valores de A com uma pequena alteração, na qual seu valor máximo é 80, mesmo que presente apenas uma vez, o valor de 20 deixa de representar os 50% dos dados, como você pode observar melhor no exemplo a seguir.



Exemplo

Pré-normalização:

A = (20; 10; 0; 40; 30; 40; 40; 10; 20; 30)

B = (20; 10; 0; 40; 30; 40; **80**; 10; 20; 30)

Pós-normalização por mínimo-máximo:

An = (0,5; 0,25; 0; 1; 0,75; 1; 1; 0,25; 0,5; 0,75)

Bn = (0,25; 0,125; 0; 0,5; 0,375; 0,5; 1; 0,125; 0,25; 0,375)

Observe que o valor "0" em A e B, que é o mínimo, corresponde a 0 também em An e Bn, assim como o valor de "40" em A e "80" em B, correspondem a 1 em An e Bn, respectivamente. Entretanto, os outros valores mudam consideravelmente entre os dois conjuntos de dados normalizados (An e Bn), mesmo que a diferença nos conjuntos originais (A e B) tenha sido de apenas um número.

Z-score

A técnica de normalização por *Z-score* tem como vantagem ser menos sensível a esses dados discrepantes e não necessitar do conhecimento prévio do valor mínimo e máximo do conjunto de dados, já que utiliza a média e o desvio-padrão da variável para recalcular os valores. A sua desvantagem é que, por depender desses fatores que mudam de acordo com a variável, esse método não produz dados normalizados com a mesma escala exata.

Nesse método, em vez de um intervalo entre 0 e 1, são esperados também números negativos, com o 0 sendo uma medida central, como -2, 0 e 2. Os novos valores são calculados com base na seguinte fórmula:

$$\frac{\text{valor} - \text{média da variável}}{\text{desvio-padrão da variável}}$$

Onde a média é o valor médio de todos os valores da variável, tal como o desvio-padrão. Quando um valor é exatamente igual à média de todos os valores, ele será normalizado como 0. Se for menor do que a média, será um número negativo e, se for maior, será positivo. O intervalo desses números é determinado pelo desvio-padrão. Se os dados não normalizados apresentarem um grande desvio-padrão, os valores normalizados estarão mais próximos de 0; caso contrário, estarão distribuídos mais próximos às extremidades. Após a transformação, os dados terão média 0 e desvio-padrão igual a 1 (OZDEMIR; SUSARLA, 2018).



Exemplo

Pré-normalização:

A = (20; 10; 0; 40; 30; 40; 40; 10; 20; 30)

B = (20; 10; 0; 40; 30; 40; **80**; 10; 20; 30)

Pós-normalização por *Z-score*:

An = (-0,3; -1; **-1,7**; **1,1**; 0,4; 1,1; 1,1; -1; -0,3; 0,4)

Bn = (-0,4; -0,8; **-1,2**; 0,5; 0,1; 0,5; **2,3**; -0,8; -0,4; 0,1)

Observe que o valor "0" em A e B, que é o mínimo, corresponde aos valores mínimos de An e Bn, tal como o valor de "40" em A e "80" em B correspondem aos valores máximos. Entretanto, é possível notar uma diferença na escala entre esses dois *datasets*, em decorrência da alteração da média e do desvio-padrão dos conjuntos de valores, ao trocar "40" por "80". Observe que, mesmo com essa alteração, não há uma mudança considerável entre os outros valores dos dois conjuntos de dados normalizados (An e Bn), mostrando que esse método "lida" melhor com discrepâncias que o anterior, de forma a não interferir tanto na distribuição.

Escala decimal

A técnica de normalização por escala decimal é aquela em que dividimos cada valor dos dados pelo valor absoluto máximo dos dados, com o objetivo de tornar a escala decimal. O valor dos dados é normalizado utilizando-se a fórmula a seguir:

$$\frac{\text{valor}}{10^m}$$

Onde o valor atual é dividido por 10 elevado a m , que é o menor inteiro que retornaria um novo valor máximo menor do que 1. Assim, se o seu valor máximo for uma unidade, m é igual a 1; se for uma dezena, m é igual a 2, e assim sucessivamente, como mostra o exemplo a seguir.



Exemplo

Pré-normalização:

U = (2; 1; 0; 4; **9**)
D = (2; **10**; 0; 4; 9)
C = (20; 10; 0; **400**; 9)
M = (**2000**; 10; 0; 400; 90)

Pós-normalização por escala decimal:

U = (0,2; 0,1; 0; 0,4; **0,9**) — como o número máximo é unidade: $\text{valor}/10^1 \rightarrow \text{valor}/10$
D = (0,2; **0,1**; 0; 0,04; 0,09) — como o número máximo é dezena: $\text{valor}/10^2 \rightarrow \text{valor}/100$
C = (0,02; 0,01; 0; **0,4**; 0,09) — como o número máximo é centena: $\text{valor}/10^3 \rightarrow \text{valor}/1000$
M = (**0,2**; 0,001; 0; 0,04; 0,009) — como o número máximo é milhar: $\text{valor}/10^4 \rightarrow \text{valor}/10000$

Observe que o valor máximo de cada conjunto de dados (em negrito) sempre será transformado em décimos, e os demais valores poderão estar em décimos, centésimos, milésimos, e outros subseqüentes sempre mantendo a proporção em relação ao valor máximo.

Qualquer que seja o método de normalização escolhido, ele pode alterar os dados originais e, por isso, é necessário salvar os parâmetros de normalização para que seja possível “destransformá-los”, caso necessário, e também para que dados futuros possam ser normalizados da mesma maneira, caso assim se deseje.

3 Aplicação da normalização de dados

Para aplicar a normalização dos dados no Python utilizando as bibliotecas Pandas, Numpy e Scikit-learn, será utilizado um conjunto de dados fictício com informações de funcionários de uma empresa, como nome, idade, anos de experiência e salário, como o exemplo do início do capítulo, que pode ser visualizado na Figura 1.

```
#Para carregar as bibliotecas  
import numpy as np  
import pandas as pd  
from sklearn import preprocessing
```

```
#Para ler o arquivo de exemplo;  
## index_col = 0 serve para utilizar a primeira coluna (nome) como rótulo  
mydata = pd.read_csv("exemplo1.csv", index_col = 0)
```

```
#Para visualizar o que há dentro do arquivo  
mydata
```

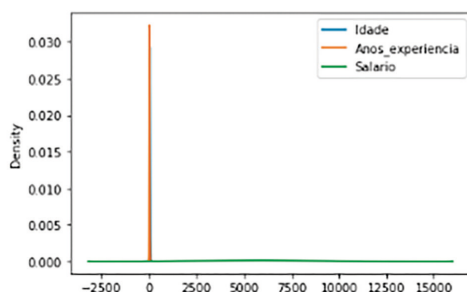
	Idade	Anos_experiencia	Salario
Alice	39	16	4800
Helena	27	4	1600
Miguel	67	47	11200
Sophia	24	2	2300
Artur	57	20	6500
Gustavo	42	19	5200
Laura	43	18	5800
Theo	51	30	6300
Pedro	44	23	6400
Jurema	45	20	8900

Figura 1. Importação das bibliotecas Numpy, Pandas e Scikit-learn, e do banco de dados que será utilizado como exemplo para os próximos passos.

Para observar a distribuição e como a diferença de escalas pode impactar na visualização dos dados em uma análise, podemos utilizar a função Pandas chamada `plot.density()`. Essa função pode ser aplicada a um `DataSet` por completo ou a uma coluna específica, como exemplificado na Figura 2. No primeiro gráfico dela, com o `DataSet` completo, é impossível observar a distribuição dos dados, dada a diferença nas escalas de idade e anos de experiência, se comparado à variável `Salario`. Já no segundo gráfico, em que essa variável não é selecionada, é possível ver a distribuição dos valores de idade (em azul) e anos de experiência (em laranja). Isso só é possível porque ambos estão contabilizados em anos, ou seja, na mesma escala.

```
#Para visualizar a distribuição dos dados  
mydata.plot.density()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x1e29e3247c8>
```



```
#Para visualizar a distribuição dos dados selecionando colunas  
mydata['Idade'].plot.density()  
mydata['Anos_experiencia'].plot.density()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x1e29e2ae488>
```

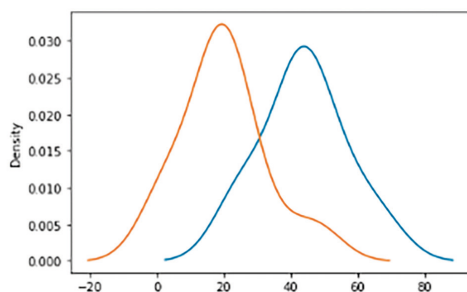


Figura 2. Utilização da função `plot.density()` para visualizar a densidade e distribuição dos dados, de forma total e de forma selecionada.

Nas normalizações a seguir, observe não só os valores transformados, mas também como esse gráfico é modificado em cada uma das transformações. Assim, a compreensão do que está acontecendo numericamente será facilitada e você poderá entender a importância das técnicas.

Aplicando o mínimo–máximo

Dentro de um `DataSet`, a função `escala.fit_transform()` transforma os dados de cada variável individualmente, de acordo com a escala definida anteriormente. No caso do método de mínimo–máximo, essa escala é definida pela função `preprocessing.MinMaxScaler(feature_range = (mínimo, máximo))`, que age individualmente em cada valor, subtraindo o valor mínimo deste e depois dividindo pelo intervalo. Esse intervalo é a diferença entre o valor máximo original e o valor mínimo original. Essa metodologia preserva a forma da distribuição original dos dados e não altera significativamente as informações incorporadas. O intervalo padrão utilizado para os novos valores da variável é de 0 a 1, como mostrado na Figura 3, mas usando o atributo `feature_scale` é possível escolher outros valores também.

Observe que agora é possível visualizar a densidade e a distribuição das três variáveis em conjunto, já que todas foram transformadas para uma mesma escala. Observe também que todas elas iniciam e terminam no mesmo ponto, tendo em vista o estabelecimento de um valor máximo e mínimo em comum. Ainda assim, a distribuição da variável idade (em azul) e anos de experiência (em laranja) se mantém a mesma que a da figura dois, embora os valores absolutos tenham mudado.

Uma variação da função `preprocessing.MinMaxScaler(feature_range = (mínimo, máximo))` é `preprocessing.MaxAbsScaler()`, que, de acordo com a documentação do Scikit-learn (2019), pode ser uma alternativa quando há *outliers*. Por esse comando se basear apenas no valor máximo, não há um deslocamento ou centralização dos dados, ou seja, não há alteração da dispersão. O valor absoluto máximo de cada variável é reescalado para 1 e todos os outros valores são reescalados com base na proporção em relação a este, como você pode observar na Figura 4.

```
#Para criar o objeto que armazenará a escala de 0 a 1
escala = preprocessing.MinMaxScaler(feature_range=(0,1))
# para transformar os valores com a nova escala
data_minmax = escala.fit_transform(mydata)
```

```
#para visualizar os dados transformados
data_minmax = pd.DataFrame(data = data_minmax)
data_minmax
```

	0	1	2
0	0.348837	0.311111	0.312500
1	0.089787	0.044444	0.000000
2	1.000000	1.000000	1.000000
3	0.000000	0.000000	0.072917
4	0.767442	0.400000	0.510417
5	0.418805	0.377778	0.375000
6	0.441880	0.355556	0.437500
7	0.627907	0.622222	0.489583
8	0.465116	0.488889	0.500000
9	0.488372	0.400000	0.760417

```
#Para visualizar a distr
data_minmax.plot.density()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x1e29ee72148>
```

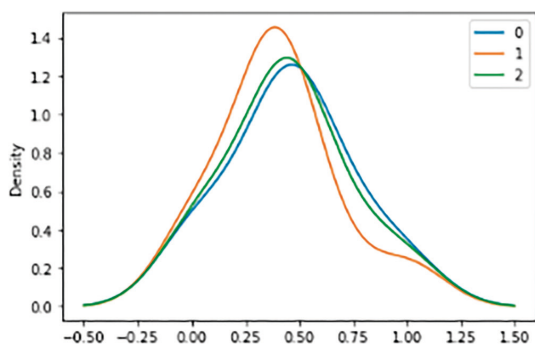


Figura 3. Exemplo de utilização do comando `preprocessing.MinMaxScaler()` para determinar a escala com base no valor mínimo e máximo e da função `fit.transform()` para aplicar a nova escala nos dados. A função `plot.density()` foi utilizada para visualizar a densidade dos dados após as transformações.

```
: #Para criar o objeto que armazenará a escala com base em 1
escala = preprocessing.MaxAbsScaler()
# para transformar os valores com a nova escala
data_max = escala.fit_transform(mydata)
```

```
: #para visualizar os dados transformados
data_max = pd.DataFrame(data = data_max)
data_max
```

```
:

```

	0	1	2
0	0.582090	0.340426	0.410714
1	0.402985	0.085106	0.142857
2	1.000000	1.000000	1.000000
3	0.358209	0.042553	0.205357
4	0.850746	0.425532	0.580357
5	0.626886	0.404255	0.484288
6	0.641791	0.382979	0.517857
7	0.761194	0.638298	0.562500
8	0.656716	0.489382	0.571429
9	0.671642	0.425532	0.794843

```
: #Para visualizar a distr
data_max.plot.density()
```

```
: <matplotlib.axes._subplots.AxesSubplot at 0x1e29e244b48>
```

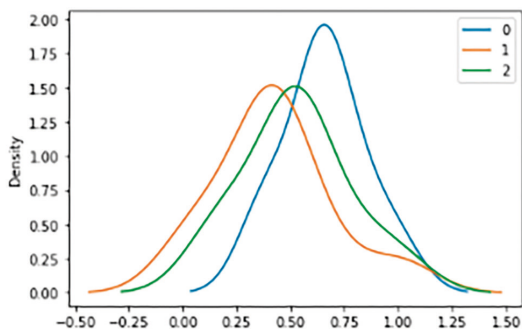


Figura 4. Utilização do comando `preprocessing.MaxAbsScaler()` para determinar a escala com base no valor máximo e da função `fit.transform()` para aplicar a nova escala nos dados. A função `plot.density()` foi utilizada para visualizar a densidade dos dados após as transformações.

No gráfico da Figura 4, a distribuição já se torna diferente do que é visto na Figura 3. Agora a linha azul, que corresponde à idade, assume uma distribuição diferente, baseada no seu valor mais alto e não na centralização de valores, como feito anteriormente. A literatura pouco fala sobre esse tipo de normalização, sendo mais usual entre esses dois casos aplicar o valor mínimo e máximo, como na função mostrada anteriormente.

Aplicando o Z-score

O *Z-Score* é aquela medida que utiliza a média e o desvio-padrão das variáveis para recalcular os novos valores. Em Python, esses novos valores podem ser obtidos com a função `preprocessing.StandardScaler(with_mean = True, with_std = True)`. Essa função calculará então a média e o desvio-padrão de cada variável (isso é indicado pelos parâmetros `with_mean = True` e `with_std = True`, que significam, respectivamente, “com média” e “com desvio-padrão”). Em seguida, a fórmula pode ser aplicada aos dados utilizando a função `fit_transform(dataset)`, onde `dataset` é o objeto onde está armazenado o seu conjunto de dados, como visto nos exemplos anteriores. Um exemplo da sua utilização para o *Z-score* pode ser visto na Figura 5.

Observe na tabela da Figura 5 que agora os valores mínimos e máximos são diferentes entre si e também diferentes de 0 e 1, respectivamente. Nesse tipo de normalização não há uma escala fixa para todo o `DataSet`, de forma que ela é determinada de acordo com os dados de cada variável. O valor zero não aparece em nenhum dos extremos, porque, nesse caso, os dados são centralizados, e o *score* representa a diferença deles em relação à média e desvio-padrão. Assim, os números próximos a zero nessa escala são apenas aqueles cujo valor original é igual à média.

No gráfico logo abaixo da tabela você pode observar uma distribuição uniforme entre os dados, semelhante ao que foi visto na Figura 3. Apesar disso, aqui os valores do eixo X variam entre o negativo e positivo, e as curvas não necessariamente começam e terminam na mesma posição.

```
#Para criar o objeto que armazenará a escala com base na média e desvio padrão
escala = preprocessing.StandardScaler(with_mean= True, with_std= True)
# para transformar os valores com a nova escala
data_zscore = escala.fit_transform(mydata)
```

```
#para visualizar os dados transformados
data_zscore = pd.DataFrame(data = data_zscore)
data_zscore
```

	0	1	2
0	-0.404572	-0.325124	-0.478708
1	-1.395381	-1.325508	-1.800874
2	1.907288	2.259198	1.989823
3	-1.843058	-1.492237	-1.338882
4	1.081811	0.008337	0.231873
5	-0.158875	-0.075029	-0.254313
6	-0.074309	-0.158394	-0.029919
7	0.588217	0.841988	0.157075
8	0.008257	0.258432	0.194474
9	0.090822	0.008337	1.129448

```
#Para visualizar a distr
data_zscore.plot.density()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x1e2a0819488>
```

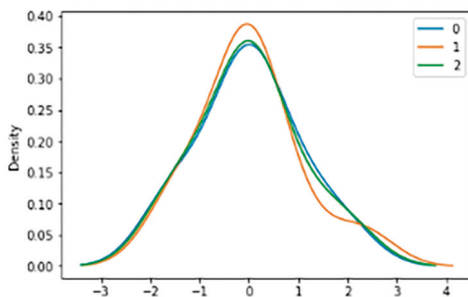


Figura 5. Utilização do comando `preprocessing.StandardScaler()` para determinar a escala com base na mediana e no desvio-padrão e da função `fit.transform()` para aplicar a nova escala nos dados. A função `plot.density()` foi utilizada para visualizar a densidade dos dados após as transformações.

Aplicando a escala decimal

Dentro dessas três bibliotecas, não existe uma função propriamente dita para aplicar a escala decimal. Entretanto, como já vimos, a partir do conhecimento da fórmula aplicada para cada tipo de normalização é possível realizá-la manualmente com a criação de uma função. Em bancos de dados com poucas variáveis, como o nosso, a criação da função não é estritamente obrigatória, já que é possível analisar individualmente cada variável que compõe o banco. Neste caso, pode-se identificar o valor máximo de cada variável por meio da função `.max()` e utilizá-lo como referência para decidir qual será o número divisor da fórmula apresentada no tópico “Escala decimal”, como exemplificado na Figura 6.

```
#Para copiar o banco para um novo objeto data_dec
data_dec = mydata
#Para descobrir o máximo de cada variável no obeto copiado
data_dec.max()
```

```
Idade           67
Anos_experiencia  47
Salario        11200
dtype: int64
```

```
#Se Idade e Anos_experiencia são maiores que 10
#e menores que 100, devem ser divididos por 100.
data_dec['Idade'] = data_dec['Idade']/100
data_dec['Anos_experiencia'] = data_dec['Anos_experiencia']/100

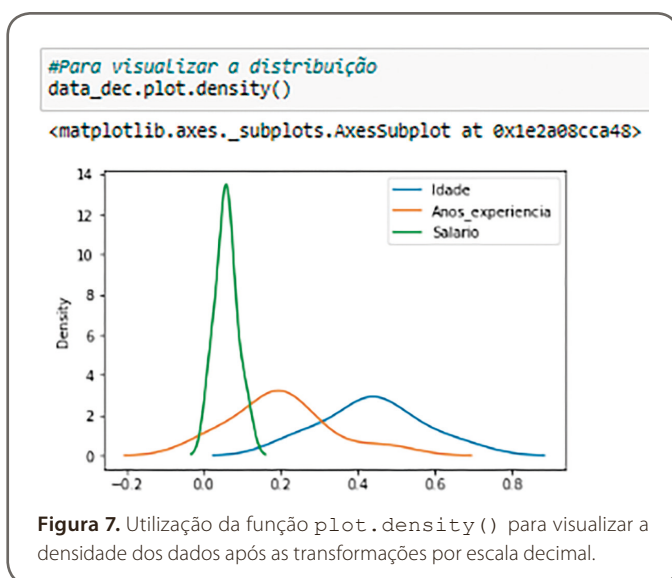
#Se Salario é maior que 10.000 e menor que 100.000
#deve ser dividido por 100.000.
data_dec['Salario'] = data_dec['Salario']/100000
```

```
data_dec
```

	Idade	Anos_experiencia	Salario
Alice	0.39	0.16	0.046
Helena	0.27	0.04	0.016
Miguel	0.67	0.47	0.112
Sophia	0.24	0.02	0.023
Artur	0.57	0.20	0.065
Gustavo	0.42	0.19	0.052
Laura	0.43	0.18	0.058
Theo	0.51	0.30	0.063
Pedro	0.44	0.23	0.064
Jurema	0.45	0.20	0.069

Figura 6. Implementação da normalização por escala decimal no Python.

É importante identificar a sensibilidade desse método a valores discrepantes (*outliers*). O valor máximo da variável `Salario` era o único com duas dezenas de milhar (11.200) e, como o divisor precisa ser escolhido com base nele, os outros números foram “penalizados”, ficando muito menores quando comparados aos valores das outras variáveis. Isso pode ser evidenciado no gráfico da Figura 7, em que você pode ver que há uma maior distorção da distribuição do `Salario` em relação aos métodos anteriores, que pode ser explicada por esse valor discrepante.



O que fazer quando há muitos *outliers*

Na presença de muitos valores discrepantes (*outliers*), é possível normalizar os dados de forma mais robusta, utilizando a função `RobustScaler()` do Scikit-learn. Essa função deve ser utilizada nos casos em que a média e a variação são altamente influenciadas pelos *outliers* (ALBON, 2018).

Esse método remove a mediana e normaliza os dados de acordo com o intervalo interquartil (IQR, do inglês *interquartile range*). O IQR é o intervalo entre o 1º quartil e o 3º quartil da distribuição de dados. Aqui a normalização também ocorre de forma independente para cada variável, em que a mediana e o IQR1 e IQR3 são armazenados para, em seguida, serem utilizados na transformação dos dados, como pode ser visto na Figura 8.

Observe também na Figura 8 a diferente disposição dos valores normalizados frente às tabelas anteriormente apresentadas e a distribuição mais uniforme da variável `Salario`, se comparada à normalização por escala decimal (Figura 7). Aqui, assim como no *Z-score*, a amplitude dos valores abrange números positivos e negativos, com um intervalo diferente para cada variável.

```
#Para criar o objeto que armazenará a escala com base no IQR
escala = preprocessing.RobustScaler(quantile_range = (25.0 , 75.0))
# para transformar os valores com a nova escala
data_robust = escala.fit_transform(mydata)
```

```
#para visualizar os dados transformados
data_robust = pd.DataFrame(data = data_robust)
data_robust
```

	0	1	2
0	-0.481538	-0.808898	-0.840580
1	-1.892308	-2.895852	-2.579710
2	2.410258	4.782809	2.985507
3	-2.000000	-3.043478	-2.173913
4	1.384815	0.088957	0.280870
5	-0.153848	-0.088957	-0.492754
6	-0.051282	-0.280870	-0.144928
7	0.789231	1.828087	0.144928
8	0.051282	0.808898	0.202899
9	0.153848	0.088957	1.652174

```
#Para visualizar a distribuição dos dados
data_robust.plot.density()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x1e2a0977e88>
```

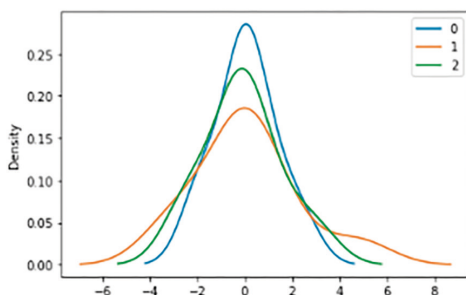


Figura 8. Utilização do comando `preprocessing.RobustScaler()` para determinar a escala com base no intervalo interquartil e da função `fit.transform()` para aplicar a nova escala nos dados. A função `plot.density()` foi utilizada para visualizar a densidade dos dados após as transformações.

O que podemos observar comparando todos os métodos apresentados é que, quando não há a presença de valores muito discrepantes, a normalização ocorre de forma mais centralizada e distribuída, não interferindo significativamente na distribuição dos dados se comparados aos valores originais. Entretanto, na presença de *outliers*, é interessante buscar por métodos que visem a minimizar a importância desses valores, como o método de *Z-Score* ou de intervalo interquartil, como foi apresentado neste último tópico. Todas essas normalizações devem ser estudadas antes de serem implementadas em um banco de dados, a fim de ser identificada a mais adequada para cada banco e tipo de análise.



Referências

AL SHALABI, L.; SHAABAN, Z.; KASASBEH, B. Data mining: a preprocessing engine. *Journal of Computer Science*, v. 2, nº. 9, p. 735–739, 2006. Disponível em: https://www.researchgate.net/profile/Luai-Shalabi/publication/26445674_Data_Mining_A_Preprocessing_Engine/links/5408236e0cf2c48563b8a311/Data-Mining-A-Preprocessing-Engine.pdf. Acesso em: 3 jul. 2020.

ALBON, C. *Machine learning with python cookbook: practical solutions from preprocessing to deep learning*. Sebastopol: O'Reilly, 2018.

OZDEMIR, S.; SUSARLA, D. *Feature engineering made easy: identify unique features from your dataset in order to build powerful machine learning systems*. Birmingham: Packt, 2018.

PATRO, S. G. K.; SAHU, K. K. *Normalization: a preprocessing stage*. 2015. Disponível em: <https://arxiv.org/ftp/arxiv/papers/1503/1503.06462.pdf>. Acesso em: 3 jul. 2020.

SCIKIT LEARN. *Preprocessing data*. 2019. Disponível em: <https://scikit-learn.org/stable/modules/preprocessing.html>. Acesso em: 3 jul. 2020.

Leituras recomendadas

CODECADEMY. *Normalization*. c2020. Disponível em: https://www.codecademy.com/articles/normalization?utm_source=rakuten&utm_medium=affiliate&utm_campaign=adgoal.net&utm_content=10-1&ranMID=44188&ranEAID=a1LgFw09t88&ranSiteID=a1LgFw09t88-lwBJxr.3bNROg4seX0Q. Acesso em: 3 jul. 2020.

JAIN, A.; NANDAKUMAR, K.; ROSS, A. Score normalization in multimodal biometric systems. *Pattern recognition*, v. 38, nº. 12, p. 2270–2285, 2005. Disponível em: http://biometrics.cse.msu.edu/Publications/Multibiometrics/JainNandakumarRoss_Score-Normalization_PR05.pdf. Acesso em: 3 jul. 2020.



Fique atento

Os *links* para *sites da web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS